OpenClaw 使用指导

> 整合橙皮书、Datawhale 教程、APIFox 教程等资源的精华内容
>
> 版本：v1.1 | 更新时间：2026-03-17

📖 目录

1. 认识 OpenClaw
2. 快速开始
3. 安装部署
4. 渠道接入
5. 配置指南
6. 多 Agent 配置
7. Skills 系统
8. 记忆系统
9. 定时任务
10. 常用命令
11. 故障排查

---

1. 认识 OpenClaw

1.1 什么是 OpenClaw?

OpenClaw 是一个**开源、自托管的 AI Agent 系统**，让 AI 从「聊天工具」变成「能自主执行任务的数字员工」。

维度	ChatGPT	OpenClaw
定位	问答顾问	数字员工
部署	云端 SaaS	本地自托管

维度	ChatGPT	OpenClaw
交互	网页对话	20+ 聊天渠道
能力	回答问题	执行任务
数据	云端存储	本地掌控

1.2 核心能力

- ✅ **多渠道连接**：Telegram、WhatsApp、飞书、钉钉、Discord 等
- ✅ **持久记忆**：记住你的偏好、项目、联系人
- ✅ **技能扩展**：通过 Skills 系统无限扩展能力
- ✅ **主动工作**：定时任务、心跳检查、自动提醒
- ✅ **数据本地**：所有数据保存在你的设备上
- ✅ **多 Agent 协作**：不同任务使用专属 Agent

1.3 架构概览

```
┌──────────────────────────────────────────────────────────┐
│                      用户聊天渠道                          │
│   Telegram / WhatsApp / 飞书 / 钉钉 / Discord ...         │
└──────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│                    Gateway 核心服务                        │
│  - 消息路由   - 会话管理   - 事件处理   - 任务调度          │
└──────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│                      Agent 系统                           │
│  main / chat / coding / 自定义 Agent                       │
└──────────────────────────────────────────────────────────┘
                        │
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│                    工具与技能                              │
│  Skills / Plugins / Tools / Memory / Cron                  │
└──────────────────────────────────────────────────────────┘
```

---

2. 快速开始

2.1 系统要求

要求	说明
Node.js	v22.0.0 或更高
操作系统	macOS / Linux / Windows (WSL2)
内存	最低 2GB，推荐 4GB+
存储	最低 1GB 可用空间

2.2 一键安装

macOS / Linux：

```
curl -fsSL https://openclaw.bot/install.sh | bash
```

Windows (PowerShell)：

```
iwr -useb https://openclaw.ai/install.ps1 | iex
```

2.3 验证安装

```
# 检查版本
openclaw --version

# 运行诊断
openclaw doctor

# 查看状态
openclaw status
```

2.4 5 分钟快速体验

```
# 1. 安装
npm i -g openclaw

# 2. 运行设置向导
openclaw onboard

# 3. 启动 Gateway
openclaw gateway start

# 4. 打开控制台
openclaw dashboard

# 5. 在浏览器中开始对话!
```

---

3. 安装部署

3.1 安装方式对比

方式	难度	适用场景
一键脚本	⭐	新手推荐
npm 手动安装	⭐⭐	熟悉 Node.js
Docker	⭐⭐	容器化部署
源码编译	⭐⭐⭐⭐	开发者

3.2 手动安装（npm）

```
# 确认 Node.js 版本
node -v  # 需要 >= 22.0.0

# 全局安装
npm install -g openclaw@latest

# 如果遇到 sharp 模块错误
SHARP_IGNORE_GLOBAL_LIBVIPS=1 npm install -g openclaw@latest
```

3.3 配置 PATH（如需要）

如果提示 `openclaw: command not found`：

```
# 找到 npm 全局路径
npm prefix -g

# 添加到 ~/.bashrc 或 ~/.zshrc
export PATH="$(npm prefix -g)/bin:$PATH"

# 生效
source ~/.bashrc
```

3.4 Docker 部署

```
docker run -d \
  --name openclaw \
  -v ~/openclaw-data:/root/.openclaw \
  -p 18789:18789 \
  openclaw/openclaw:latest
```

3.5 版本更新

```
# 更新到稳定版（推荐）
openclaw update --channel stable

# 更新到测试版
openclaw update --channel beta

# 更新到开发版
openclaw update --channel dev
```

---

4. 渠道接入

4.1 渠道难度对比

梯队	平台	耗时	说明
第一梯队	Telegram、QQ	5-10 分钟	最简单，零门槛
第二梯队	Discord、飞书	15-20 分钟	文档齐全
第三梯队	WhatsApp、Slack、钉钉	25-40 分钟	配置较多
第四梯队	iMessage、微信	1 小时+	需额外条件

4.2 Telegram 接入（推荐入门）

步骤 1：创建 Bot

1. 在 Telegram 搜索 `@BotFather`
2. 发送 `/newbot` 命令
3. 设置 bot 名称（以 `bot` 结尾）
4. 保存返回的 Bot Token

步骤 2：配置 OpenClaw

编辑 `~/.openclaw/openclaw.json`：

```
{
  "channels": {
    "telegram": {
      "enabled": true,
      "botToken": "YOUR_BOT_TOKEN",
      "dmPolicy": "pairing",
      "allowFrom": ["tg:YOUR_USER_ID"]
    }
  }
}
```

步骤 3：启动并配对

```
# 重启 Gateway
openclaw gateway restart

# 在 Telegram 给 bot 发消息
# 会收到一个配对验证码
```

4.3 飞书接入

```
# 运行配置向导
openclaw configure --section feishu
```

需要在飞书开放平台创建企业自建应用，获取 App ID 和 App Secret。

4.4 多渠道配置

可以同时配置多个渠道：

```
{
  "channels": {
    "telegram": { "enabled": true, "botToken": "..." },
    "feishu": { "enabled": true, "appId": "...", "appSecret": "..." },
    "discord": { "enabled": true, "botToken": "..." }
  }
}
```

---

5. 配置指南

5.1 配置文件位置

文件	路径	说明
主配置	`~/.openclaw/openclaw.json`	Gateway、渠道、模型配置
Agent 配置	`~/.openclaw/agents/*/agent/`	各 Agent 独立配置
工作区	`~/.openclaw/workspace-*/`	各 Agent 工作空间

5.2 模型配置

编辑 `~/.openclaw/openclaw.json`：

```
{
  "models": {
    "mode": "merge",
```

```
    "providers": {
      "bailian": {
        "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
        "apiKey": "sk-YOUR_API_KEY",
        "api": "openai-completions",
        "models": [
          {
            "id": "qwen3.5-plus",
            "name": "qwen3.5-plus",
            "contextWindow": 1000000,
            "maxTokens": 65536
          }
        ]
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "bailian/qwen3.5-plus"
      }
    }
  }
}
```

5.3 常用模型推荐

提供商	模型	适用场景
阿里云	qwen3.5-plus	通用任务，性价比高
阿里云	qwen3-coder-plus	编程、代码生成
智谱	glm-5	中文任务优化
月之暗面	kimi-k2.5	长文本处理

5.4 Gateway 配置

```
{
  "gateway": {
    "port": 18789,
    "mode": "local",
```

```
    "bind": "lan",
    "auth": {
      "mode": "token",
      "token": "your-secure-token"
    },
    "controlUi": {
      "allowedOrigins": ["http://192.168.1.100:18789"],
      "allowInsecureAuth": true
    }
  }
}
```

5.5 认证模式

Token 认证（推荐 API 集成）：

```
{
  "gateway": {
    "auth": {
      "mode": "token",
      "token": "your-token"
    }
  }
}
```

密码认证（推荐 Web UI）：

```
{
  "gateway": {
    "auth": {
      "mode": "password"
    }
  }
}
```

---

6. 多 Agent 配置

6.1 什么是多 Agent?

OpenClaw 支持运行多个独立的 Agent 实例，每个 Agent 可以：

- 使用不同的模型配置
- 拥有独立的工作空间和记忆
- 专注特定类型的任务
- 独立配置 Skills 和工具

6.2 内置 Agent 类型

Agent	用途	推荐模型
main	主对话 Agent，处理日常任务	qwen3.5-plus
chat	纯聊天 Agent，轻松对话	qwen3.5-plus
coding	编程专用 Agent，代码生成/审查	qwen3-coder-plus / glm-5
analysis	数据分析 Agent	qwen3.5-plus

6.3 创建自定义 Agent

```
# 创建新 Agent
openclaw agents create my-agent

# 设置专属模型
openclaw agents config my-agent --model glm-5

# 配置工作区
openclaw agents workspace my-agent --path ~/projects/my-agent

# 启动 Agent
openclaw agents start my-agent
```

6.4 为不同任务分配 Agent

任务类型	推荐 Agent	调用方式
日常对话、问答	main	默认
写代码、Debug	coding	`/agent coding`
数据分析、报表	analysis	`/agent analysis`

任务类型	推荐 Agent	调用方式
创意写作	chat	`/agent chat`
专业领域任务	自定义 Agent	`/agent <name>`

6.5 Agent 路由配置

方式 1：命令前缀

```
/agent coding 帮我写一个 Python 脚本
/agent analysis 分析这个数据
```

方式 2：自动路由（基于关键词）

```
{
  "routing": {
    "rules": [
      {
        "keywords": ["代码", "编程", "python", "javascript"],
        "agent": "coding"
      },
      {
        "keywords": ["分析", "数据", "图表"],
        "agent": "analysis"
      }
    ]
  }
}
```

方式 3：渠道绑定

```
{
  "channels": {
    "telegram": {
      "defaultAgent": "main"
    },
    "discord": {
      "defaultAgent": "chat"
    }
```

```
  }
}
```

6.6 多 Agent 最佳实践

✅ 推荐做法：

- 为高频任务创建专用 Agent
- 编程任务使用独立 Agent（避免上下文污染）
- 为不同项目创建独立 Agent 实例
- 定期清理不活跃 Agent

❌ 避免：

- 创建过多 Agent（建议 ≤5 个）
- 所有 Agent 使用相同模型（浪费资源）
- 忽略 Agent 间的记忆隔离

6.7 查看和管理 Agent

```
# 列出所有 Agent
openclaw agents list

# 查看 Agent 状态
openclaw agents status my-agent

# 查看 Agent 配置
openclaw agents config my-agent --show

# 停止 Agent
openclaw agents stop my-agent

# 删除 Agent
openclaw agents delete my-agent
```

6.8 多 Agent 架构示例

```
┌──────────────────────────────────────────────────────────────┐
│                          用户消息                            │
└──────────────────────────┬───────────────────────────────────
                           │
```

```
                    ▼
┌──────────────────────────────────────────────────────────┐
│                路由层（根据规则分发）                      │
│   关键词匹配 / 命令前缀 / 渠道绑定                        │
└──────────────────────────────────────────────────────────┘
          │              │              │
          ▼              ▼              ▼

    ┌──────────┐    ┌──────────┐    ┌──────────┐
    │   main   │    │  coding  │    │ analysis │
    │  Agent   │    │  Agent   │    │  Agent   │
    └──────────┘    └──────────┘    └──────────┘
         │              │              │
         ▼              ▼              ▼

    ┌──────────┐    ┌──────────┐    ┌──────────┐
    │ 通用记忆 │    │ 代码技能 │    │ 数据工具 │
    │ Qwen3.5  │    │ Codex    │    │ Pandas   │
    └──────────┘    └──────────┘    └──────────┘
```

---

7. Skills 系统

7.1 什么是 Skills?

Skills 是 OpenClaw 的插件系统，让 AI 能够：

- 访问外部 API（天气、新闻、股票）
- 操作本地资源（文件、摄像头）
- 执行特定任务（GitHub、日历、邮件）

7.2 安装 Skills

```
# 从 ClawHub 安装
openclaw skill install <skill-slug>

# 搜索 Skills
openclaw skill search <keyword>

# 列出已安装
openclaw skill list

# 更新 Skills
openclaw skill update
```

7.3 热门 Skills 推荐

Skill	功能	安装命令
weather	天气预报	`skill install weather`
github	GitHub 操作	`skill install github`
news-aggregator	新闻聚合	`skill install news-aggregator`
pinchtab-browser	浏览器自动化	`skill install pinchtab-browser`

7.4 创建自定义 Skill

```
# 创建 Skill 目录
mkdir -p ~/.openclaw/workspace-main/skills/my-skill

# 创建 SKILL.md
cat > ~/.openclaw/workspace-main/skills/my-skill/SKILL.md << 'EOF'
# my-skill

## Description
我的自定义技能

## Usage
/MyCommand [arguments]
EOF
```

---

8. 记忆系统

8.1 记忆类型

类型	文件	说明
长期记忆	`MEMORY.md`	curated 重要信息
每日记忆	`memory/YYYY-MM-DD.md`	原始日志
身份	`IDENTITY.md`	AI 人格设定

类型	文件	说明
用户	`USER.md`	用户信息
灵魂	`SOUL.md`	行为准则

8.2 记忆文件位置

```
~/.openclaw/workspace-main/
├── MEMORY.md              # 长期记忆
├── IDENTITY.md            # 身份设定
├── USER.md                # 用户信息
├── SOUL.md                # 行为准则
├── AGENTS.md              # 工作区说明
├── TOOLS.md               # 工具备注
├── HEARTBEAT.md           # 心跳任务
└── memory/
    ├── 2026-03-16.md      # 每日记忆
    └── 2026-03-15.md
```

8.3 搜索记忆

```
# 语义搜索
openclaw memory search "上次提到的项目"

# 搜索特定日期
openclaw memory search --date 2026-03-15 "会议"
```

8.4 记忆维护

建议定期（每周）审查 MEMORY.md：

- 移除过时信息
- 添加新的决策和上下文
- 整理每日记忆中的精华

---

9. 定时任务

9.1 Cron 基础

```
# 添加定时任务
openclaw cron add --schedule "0 9 * * *" --message "早上好! 今日待办: "

# 列出任务
openclaw cron list

# 运行任务
openclaw cron run <job-id>

# 删除任务
openclaw cron remove <job-id>
```

9.2 Cron 表达式

表达式	说明
`0 9 * * *`	每天 9:00
`0 9 * * 1`	每周一 9:00
`*/30 * * * *`	每 30 分钟
`0 0 1 * *`	每月 1 日 0:00

9.3 心跳任务

编辑 `HEARTBEAT.md`：

```
# HEARTBEAT.md

- [ ] 检查未读邮件
- [ ] 查看日历（24 小时内）
- [ ] 检查天气（如需外出）
- [ ] 审查记忆文件（每周）
```

9.4 实用 Cron 示例

每日新闻简报：

```
openclaw cron add \
  --name "daily-news" \
  --schedule "0 8 * * *" \
  --message "执行 news-aggregator 技能，发送每日新闻"
```

每周备份：

```
openclaw cron add \
  --name "weekly-backup" \
  --schedule "0 2 * * 0" \
  --message "备份 workspace 和 memory 目录"
```

---

10. 常用命令

10.1 Gateway 管理

```
openclaw gateway start      # 启动
openclaw gateway stop       # 停止
openclaw gateway restart    # 重启
openclaw gateway status     # 状态
openclaw gateway install    # 安装为系统服务
openclaw gateway uninstall  # 卸载服务
```

10.2 配置管理

```
openclaw onboard            # 初始设置向导
openclaw configure          # 配置渠道
openclaw doctor             # 健康检查
openclaw status             # 查看状态
openclaw dashboard          # 打开 Web 控制台
openclaw tui                # 终端界面
```

10.3 模型管理

```
openclaw models status      # 模型状态
openclaw models list        # 列出模型
```

```
openclaw models add         # 添加模型
```

10.4 渠道管理

```
openclaw channels list      # 列出渠道
openclaw channels add       # 添加渠道
openclaw channels remove    # 移除渠道
```

10.5 Agent 管理

```
openclaw agents list            # 列出 Agent
openclaw agents create <name>   # 创建 Agent
openclaw agents config <name>   # 配置 Agent
openclaw agents start <name>    # 启动 Agent
openclaw agents stop <name>     # 停止 Agent
openclaw agents delete <name>   # 删除 Agent
```

10.6 记忆操作

```
openclaw memory search <query>   # 搜索记忆
openclaw memory list             # 列出记忆文件
openclaw memory edit             # 编辑记忆
```

10.7 会话管理

```
openclaw sessions list           # 列出会话
openclaw sessions history <key>  # 查看历史
openclaw sessions send           # 发送消息到会话
```

---

11. 故障排查

11.1 常见问题

问题 1： `openclaw: command not found`

```
# 找到 npm 全局路径
npm prefix -g

# 添加到 PATH
export PATH="$(npm prefix -g)/bin:$PATH"

# 添加到 ~/.bashrc 永久生效
echo 'export PATH="$(npm prefix -g)/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

问题 2：Gateway 无法启动

```
# 检查端口占用
lsof -i :18789

# 查看日志
journalctl --user -u openclaw-gateway.service -n 50

# 重启服务
systemctl --user restart openclaw-gateway.service
```

问题 3：Telegram 收不到消息

```
# 检查 Bot Token 是否正确
cat ~/.openclaw/openclaw.json | grep botToken

# 检查 offset 是否更新
cat ~/.openclaw/telegram/update-offset-*.json

# 查看 Gateway 日志
journalctl -t gateway-start.sh -n 30 | grep telegram
```

问题 4：模型 API 调用失败

```
# 检查 API Key
openclaw models status

# 测试连接
curl -H "Authorization: Bearer YOUR_API_KEY" \
  https://api.example.com/v1/models
```

```
# 检查网络
ping api.example.com
```

11.2 日志查看

```
# Gateway 日志
journalctl --user -u openclaw-gateway.service -f

# 最近 50 条
journalctl --user -u openclaw-gateway.service -n 50

# 特定时间段
journalctl --user -u openclaw-gateway.service \
  --since "2026-03-16 10:00:00" --until "2026-03-16 12:00:00"
```

11.3 诊断命令

```
# 完整诊断
openclaw doctor

# 检查系统
openclaw status

# 检查配置
openclaw config list
```

11.4 重置与恢复

```
# 重置配置（谨慎使用）
openclaw onboard --reset

# 重新安装 Gateway
openclaw gateway uninstall
openclaw gateway install

# 清除缓存
rm -rf ~/.openclaw/browser
rm -rf ~/.openclaw/cron
```

附录

A. 资源链接

资源	链接
官方文档	https://docs.openclaw.ai
GitHub	https://github.com/openclaw/openclaw
ClawHub	https://clawhub.com
Discord	https://discord.com/invite/clawd

B. 版本信息

- 文档版本：v1.1
- 适用 OpenClaw 版本：v2026.3.11+
- 最后更新：2026-03-17

C. 贡献者

本文档整合自：

- OpenClaw 橙皮书（花叔）
- Datawhale OpenClaw 教程
- APIFox OpenClaw 使用指南
- OpenClaw 官方文档

如有问题或建议，欢迎反馈到 OpenClaw 社区。